to need *blocks* for today's project." Before the two of you jump into the fray, you need to understand the nuances of blocks. You agree to spend the morning talking about blocks, putting off today's project until after lunch.

Today's Roadmap

On a sheet of paper, Bill lists the things he wants to cover:

- A review of the basics of blocks
- An overview of *scopes* and how you can carry variables through scopes by using blocks as *closures*
- How you can further manipulate scopes by passing a block to instance_eval
- How you can convert blocks into *callable objects* that you can set aside and call later, such as Procs and lambdas

You start with the first point—a quick review of the basics. (If you already know the basics of Ruby blocks, you can skip straight to *Blocks Are Closures*, on page 77.)

The Basics of Blocks

Do you remember how blocks work? Here is a simple example to refresh your memory:

blocks/basics_failure.rb

```
def a_method(a, b)
  a + yield(a, b)
end

a_method(1, 2) {|x, y| (x + y) * 3 }  # => 10
```

You can define a block with either curly braces or the do…end keywords. A common convention is to use curly braces for single-line blocks and do…end for multiline blocks.

You can define a block only when you call a method. The block is passed straight into the method, and the method can call back to the block with the yield keyword.

Optionally, a block can have arguments, like x and y in the previous example. When you yield to the block, you can provide values for its arguments, just like you do when you call a method. Also, like a method, a block returns the result of the last line of code it evaluates.

Within a method, you can ask Ruby whether the current call includes a block. You can do that with the Kernel#block_given? method: